Languge Models: we focus on Transforme-based models which have following 3-types:

1) **Encoder-only**: Given text, focus on represeutation-learning for different tasks.
2) **Decoder-only**: Predict next token, given all previous tokens.
3) **Encoder-Decoder**: For Sequence-to-Sequence tasks e.g. machine-translation.

We consider here Decoder-Only models, common example is GPT

Decoder Models: These are auto-regressive models which predict the conditional probabilities (distributions) of each token, given all the previous tokens:

$$P_\theta(x_{1:T}) = \prod_{t=1}^{T-1} P_\theta(x_{t+1} \mid x_{\le t})$$

Model at position t predicts the next token x_{t+1}

How is it done?

V = Vocabulary Size e.g. V = 50000
T = Sequence length
B = Batch Size
d = Embedding Dimension
h = # of Attention heads
$d_h = \frac{d}{h}$ = Per head dimension
$t \in [1, 2, \cdots, T]$ = Token index
x_t = t-th token
ℓ = layer index
L = Total # of layers
d_{ff} = Feed-forward width = 4d or 8d

① Text Input → Tokenization → Embeddings

X (Raw Text sequence) → Tokenizer (BPE/Unigram) → $(x_1, x_2, \cdots, x_T)$

For text generation, tokenizer should be invertible

where $x_t \in \{0, 1, \cdots, V-1\}$

special tokens: <bos>, <eos>, <Pad>

x_t (t-th Token) → Embedding Lookup → $E_t = W_E[x_t]$ (d×1)

W_E : V×d

Look-up operation

$\Rightarrow E$: stacked over t : $\mathbb{R}^{T \times d}$

W_E : Learnable token embedding matrix

NTP with Teacher Forcing: $x_{\le t}$, x_{t+1}, x_{t+2}, ..., x_T

E → Positional Embedding (RoPE or learnable) → $H^{(0)} = E + P$ (if learnable)

T×d

② **Transformer Layers**:

$H^{(0)}$ (T×d) → Layer 1 → $H^{(1)}$ → ... → $H^{(\ell-1)}$ → " ℓ " → $H^{(\ell)}$ → ... → " L " → $H^{(L)}$ (T×d) → $LN(H^{(L)})$ → $\bar{H}$ (T×d)

For ℓ-th layer: $\tilde{H} = LN(H^{(\ell-1)})$ (T×d)

for $i = 1, 2, \cdots, h$

$$\left.\begin{aligned} Q_i &= \tilde{H} W_{Q,i} \\ K_i &= \tilde{H} W_{K,i} \\ V_i &= \tilde{H} W_{V,i} \end{aligned}\right] \in \mathbb{R}^{T \times d_h}$$

$$A_i = \text{Softmax}\left(\frac{Q_i K_i^T}{\sqrt{d_h}} + M\right) \Rightarrow T \times T$$

$$M_{t,u} = \begin{cases} 0 & u \le t \\ -\infty & u > t \end{cases}$$

Causal Mask

$O_i = A_i V_i$ (T×d_h)

$O = \text{concat}(O_1, O_2, \cdots, O_h)$ (T×d)

Project back: $U = O W_O$ ($T \times d$ = $T \times d$ · $d \times d$) $\Rightarrow$ $H^{(l-1)}_{attn} + U = H^{(l-1)}$ ($T \times d$)

MLP: $\hat{H} = LN\left(H^{(l-1)}_{attn}\right)$

$$H^{(l)} = H^{(l-1)}_{attn} + MLP(\hat{H})$$ ($T \times d$ = $T \times d$ + $T \times d$)

KV-cache: Each layer stores K_i, V_i for previous steps. At step t x_t is projected to Q_t, K_t, V_t & K_t, V_t are appended to cache.

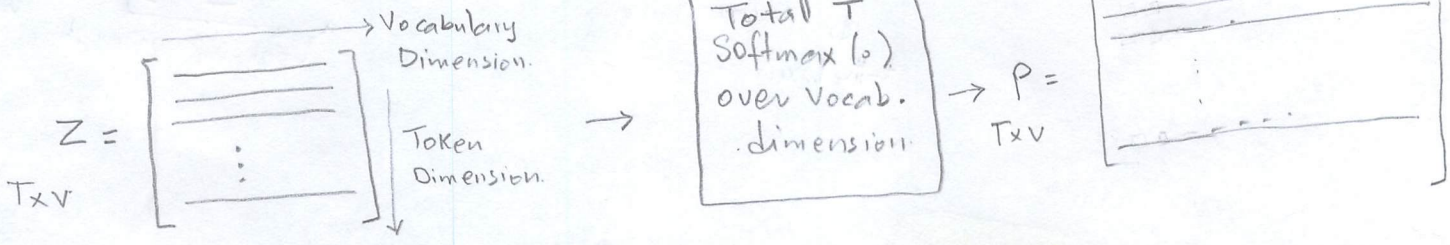
③ Final Layer: After L blocks, $H^{(L)} \in \mathbb{R}^{T \times d}$, followed by last layer-norm, we get: $\bar{H} = LN(H^{(L)})$ ($T \times d$)

Final unembedding is performed to get Logits:

Logits: $Z = \bar{H} W_E^T$ ($T \times V$ = $T \times d$ · $d \times V$) OR $Z = \bar{H} W_U$ ($T \times V$ = $T \times d$ · $d \times V$)

Tied Embedding → Reuse W_E

Learn W_U



$$P_{t,v} = \frac{e^{Z_{t,v}}}{\sum_{v'} e^{Z_{t,v'}}}$$ → One row of P matrix : t = fixed, $v \in [1, 2, \ldots V]$

Loss computation:

For each element at position t: [Input context = $[x_1, x_2, \ldots, x_t]$; Target label = x_{t+1}]

The probability assigned by model to true token (target label) = $P_{t,y_t} = \frac{e^{Z_{t,y_t}}}{\sum_{v'} e^{Z_{t,y_t}}}$

where y_t = True token (x_{t+1}) ID

= Not a value but an index telling us which probability to evaluate.

Ex. $T=4, V=6$, $Z = \begin{bmatrix} z_{1,1} & \cdots & z_{1,6} \\ \vdots & & \vdots \\ z_{4,1} & \cdots & z_{4,6} \end{bmatrix}$ → each row represents one time step t i.e. one position in sequence

↓ each column is one vocabulary token.

At any t, true next token (x_{t+1}) can take any index from $v = 1, \ldots, V$

Let target sequence has true next token indices as:

$$y = [y_1 \; y_2 \; y_3 \; y_4] = [3 \quad 5 \quad 2 \quad 4]$$

$$L_{CE} = -s_j + \log\left(\sum_{i=1}^{K} e^{s_i}\right) \quad \text{if } x \in \text{class-}j$$

⇒ At $t=1$, true next token has ID = 3

⇒ " $t=2$, " " " " " " = 5 & So on.

Depending on true next token ID, we pick the right logit at t:

$$\ell_t = -z_{t,y_t} + \log \sum_{v'=1}^{V} e^{z_{t,v'}} = -\log P_\theta(x_{t+1} \mid x_{\le t})$$

Logit from row = t & column have index = True label ID = y_t

So, for $y = [3 \quad 5 \quad 2 \quad 4]$, we use.

Teacher Forcing: Feed model true previous token when predicting next token

$$\ell_1 = -z_{1,3} + \log \Sigma_{v'} \text{''}$$
$$\ell_2 = -z_{2,5} + \log \Sigma_{v'} \text{''}$$
$$\ell_3 = -z_{3,2} + \log \Sigma_{v'} \text{''}$$
$$\ell_4 = -z_{4,4} + \log \Sigma_{v'} \text{''}$$

$$Z = \begin{matrix} t=1 \\ t=2 \\ t=3 \\ t=4 \end{matrix} \begin{bmatrix} z_{1,1} & z_{1,2} & \mathbf{z_{1,3}} & z_{1,4} & z_{1,5} & z_{1,6} \\ z_{2,1} & z_{2,2} & z_{2,3} & z_{2,4} & \mathbf{z_{2,5}} & z_{2,6} \\ z_{3,1} & \mathbf{z_{3,2}} & z_{3,3} & z_{3,4} & z_{3,5} & z_{3,6} \\ z_{4,1} & z_{4,2} & z_{4,3} & \mathbf{z_{4,4}} & z_{4,5} & z_{4,6} \end{bmatrix}$$

$$\Rightarrow L = \frac{1}{T-1} \sum_{t=1}^{T-1} [\ell_t] = \frac{1}{T-1} \sum_{t=1}^{T-1} \left(-z_{t,y_t} + \log \sum_{v=1}^{V} e^{z_{t,v}}\right)$$

In reality, we process the data in batches: # of batches = B

Batch Index = $b = 1, 2, \ldots, B$

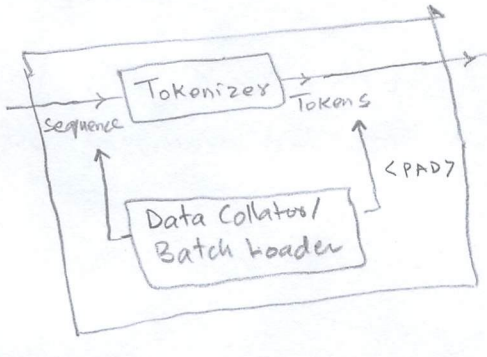
⇒ $Z : B \times T \times V$

$P : B \times T \times V$

→ Every batch b must have same sequence length = T. However, not all sequences are naturally of same length.

Ex. B = 3

	Text	True Length
b = 1	"The cat sat on the mat"	6
b = 2	"I Love AI"	3
b = 3	"Transformers are large language models"	5



So, we pad all sequences to a maximum length $T = 8$ as:

<bos> ___ Text tokens ___ <eos>

b=1 : [<bos>, <the>, <cat>, <sat>, <on>, <the>, <mat>, <eos>]

b=2 : [<bos>, <I>, <Love>, <AI>, <eos> <PAD>, <PAD>, <PAD>]

b=3 : [<bos>, <Tranformers>, <are>, <large>, <language> <models> <eos>, <PAD>]

Now, we define a **binary mask** which excludes these <PAD> tokens from computations:

$$m_{b,t} = \begin{cases} 1 & \text{if position } t \text{ has real token} \\ 0 & \text{if " " " PAD "} \end{cases}$$

So, loss over batches becomes:

$$L(\theta) = -\frac{1}{\underbrace{\sum_{b=1}^{B}\sum_{t=1}^{T-1} m_{b,t}}_{N_{valid}}} \sum_{b=1}^{B}\sum_{t=1}^{T-1} m_{b,t} \log P_\theta\left(x_{b,t+1} \middle| x_{b,\le t}\right)$$

N_{valid} = count of valid tokens over all batches.

e.g. $B = 2$, $T = 5$, Two Masks: $m_1 = [1,1,1,1,0]$, $m_2 = [1,1,1,0,0]$

of valid tokens = $\sum m_{b,t} = 4 + 3 = 7$

$$L(\theta) = -\frac{1}{7}\left[\sum_{t=1}^{4} \log P_\theta\left(x_{1,t+1} \middle| x_{1,\le t}\right) + \sum_{t=1}^{3} \log P_\theta\left(x_{2,t+1} \middle| x_{2,\le t}\right)\right]$$

We can show: $\frac{\partial L}{\partial z_{b,t,v}} = \frac{m_{b,t}}{N_{valid}}\left(P_{b,t,v} - 1[v = y_{b,t}]\right)$

Softmax Layer – Bottleneck: In decoder-based LLM, output layer Softmax operates independently along V-dimension. If $d = 4096$, $V = 50000$, $T = 1024$, $B = 8$ $\Rightarrow$ last-layer has 200 million parameters.

- 50000-dimensional Softmax per token $\rightarrow$ GPUs do this in parallel for millions of tokens at once, using optimized kernels. Other approximation tricks are also employed.

Pre-Training LLM: Big models; need some planning dials before/during training.

Scaling Laws: Empirical Power-Law relationships showing how LLM's loss & many downstream metrics, improve as we scale:

1- Model Size : N
2- Training tokens (data): D
3- Compute/FLOPS : C

] How far scale alone can get us & how to allocate compute C bw Parameters N & tokens D.

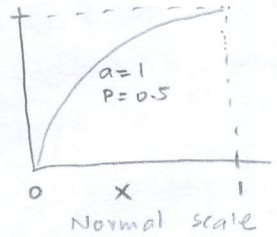
"The test loss of this LLM varies as a Power-law with respect to no. of parameters"

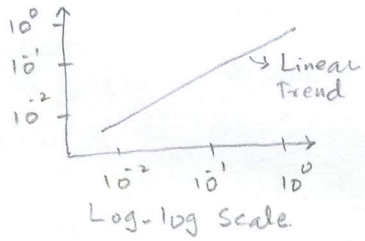
↳ ? – There is a relationship b/w two quantities : Loss & N s.t. change in one quantity produces a relative scale-invariant change in other

Ex. x & y are two quantities; Powe-law is expressed as :

$$y = a\,x^{p} \quad (1)$$

– a & P affect shape of plot





Power-law = Linear Trend on log-scale

↓

One quantity varies as a Power of another

↓

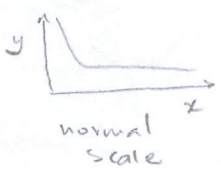
Fitting form:

$$f(x) = a\,x^{-k}$$

Inverse_version:

$$y = a\left(\frac{1}{x}\right)^{p} \rightarrow$$

Same as (1) with a negative exponent for P





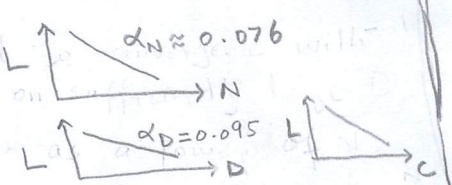
For neural-nets:

$$L(P_i) = \sum_{i=1}^{n} \alpha_i\, P_i^{-\beta_i}$$

L = Loss
P_i = performance metric → N, D, C etc
β_i = Scaling Parameter
α_i = fitting "

(1) The Foundation: Kaplan Laws: Loss L scales as a Power-law when limited by individual factors: N, D, C

- $L(N) = E + A\,N^{-\alpha_N} \Rightarrow L(N) \propto N^{-\alpha_N}$
- $L(D) = E + B\,N^{-\alpha_D} \Rightarrow L(N) \propto D^{-\alpha_D}$
- $L(C) = E + F\,C^{-\alpha_C} \Rightarrow L(C) \propto C^{-\alpha_C}$



Combined law when N & D are varied simultaneously:

$$L(N, D) \propto \left(\frac{1}{N^{\alpha_N}} + \frac{1}{D^{\alpha_D}}\right)$$

optimal data-to-parameter ratio: To avoid being bottlenecked, the model should be scaled s.t. performance penalty from small N is ≈ penalty from small D

⇒ This occurs when two terms in combined law are balanced:

$$\frac{1}{N^{\alpha_N}} \propto \frac{1}{D^{\alpha_D}} \Rightarrow D \propto N^{\frac{\alpha_N}{\alpha_D}} \Rightarrow \boxed{D \propto N^{0.74}} \rightarrow \text{Sub-linear}$$

↓

Data should grow slower than N. ⇒ Big models are more important than big data.

↓

for a fixed compute budget C.

Compute-scaling:

Using fixed compute constraint : $C \propto N \cdot D$, how both N & D should grow with C :

- $N \propto C^{0.73}$
- $D \propto C^{0.27}$

② The unified view: Chinchilla Laws: The Chinchilla paper challenged Kaplan optimal ratio, $D \propto N^{0.73}$, by showing models were actually under-trained. They proposed:

$$L(N, D) = E + \frac{A}{N^{\alpha_N}} + \frac{B}{D^{\alpha_D}} \quad \text{with} \quad \alpha_N \approx \alpha_D \approx 0.34$$

Total computational cost for given N & D $= C \approx 6 \cdot N \cdot D \rightarrow$ For a fixed cost, N & D are inversely related as ND should be const.

Find N & D that minimize $L(N, D)$ s.t. $C = 6ND \Rightarrow$ This happens when:

$$\frac{A}{N^{\alpha_N}} \approx \frac{B}{D^{\alpha_D}} \Rightarrow \text{& since } \alpha_N \approx \alpha_D$$

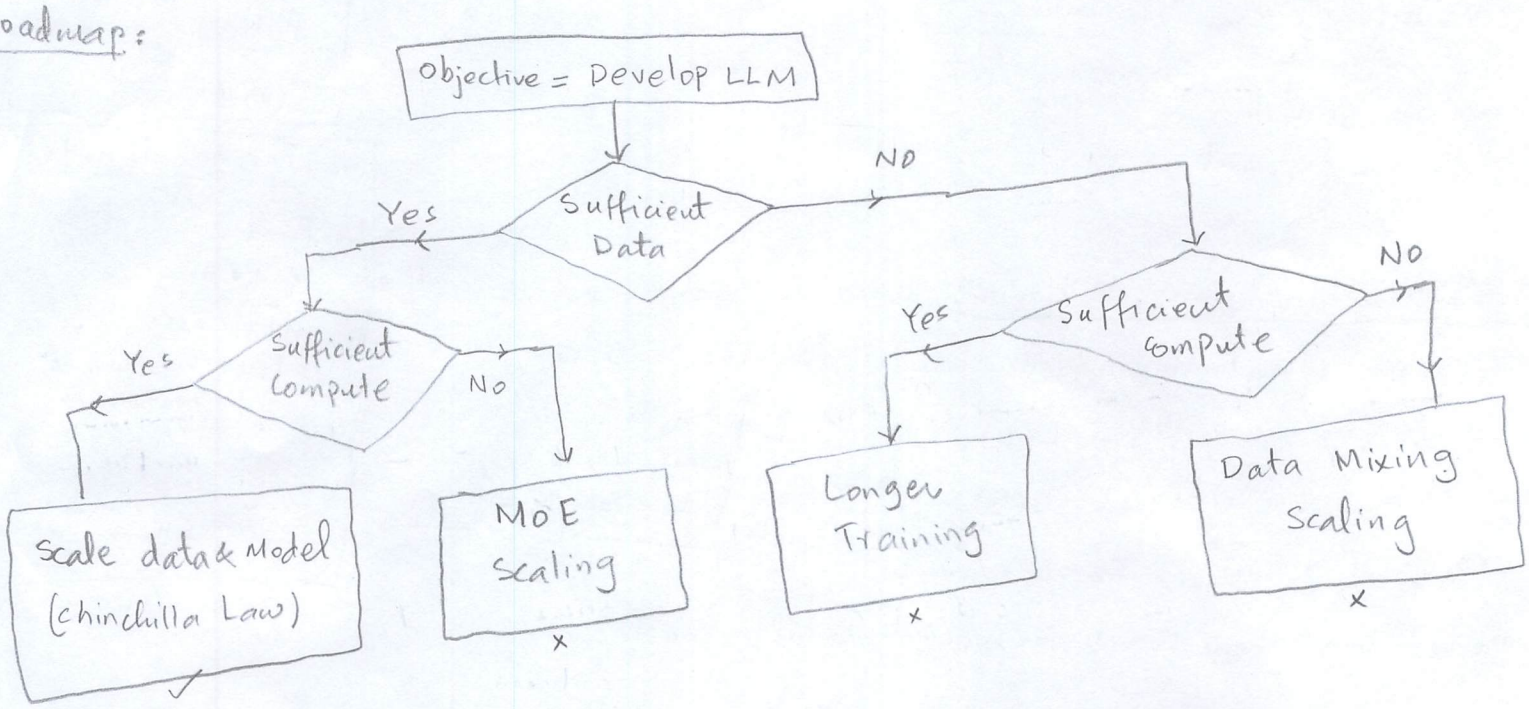
$$\Rightarrow \left(\frac{D}{N}\right)^{\alpha} \approx \frac{B}{A} = \text{const} \Rightarrow \boxed{D \propto N} \rightarrow$$

For fixed C, to get lower loss we must scale N & D at approx. same linear rate

⇒ So, C can be divided as $N \propto C^{0.5}$ & $D \propto C^{0.5}$

Rule of thumb: Budget 20 train-tokens for 1 parameter

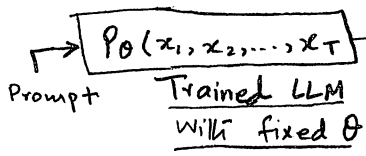
Roadmap:



→ scaling-law research has expanded to other dimensions of LLM as well:

- Fine-tuning / alignment stage laws
- Inference-time laws
- Multimodal LLM laws

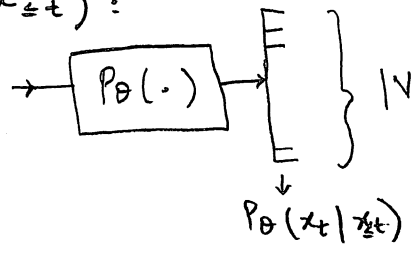
How to decode with causal LLM?



Select a sequence of tokens as model is generating them =

Decoding | Sampling

At each step t, model give a probability distribution $P_\theta(x_t | x_{<t})$:



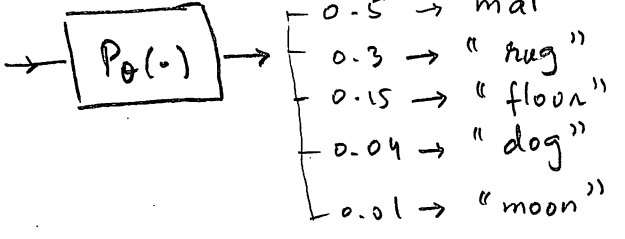
→ which x_t to choose?

Ex. $V = [\text{"mat"}, \text{"rug"}, \text{"floor"}, \text{"dog"}, \text{"moon"}] \Rightarrow |V| = 5$

Let $x_{<t}$: ["The", "cat", "sat", "on", "the"]

$x_{t+1} = ?$

Also, suppose we get predicted prob. for x_{t+1} as:



$x_{t+1} = ?$ → Balance generating a high-probability (fluent & coherent) sequence with generating a diverse & non-repetitive sequence.

① **Greedy Sampling**: Always select the token with the highest prob.

$$x_{t+1} = \arg\max_x P_\theta(x_t | x_{<t})$$

Ex.: In our example, $x_{t+1} =$ "mat"

Problem: Myopic scheme which produces locally good but globally sub-optimal sequence.

② **Naive Sampling with temperature ρ**: Pick randomly from all. ↓ Reflects model uncertainty:

- "mat" → 0.5
- "rug" → 0.3

$\rho = 1 \rightarrow P_\theta(x_t | x_{<t})$ remains same

$\rho = 0.5 \rightarrow$ " modified to [0.75, 0.2, 0.04, 0.008, 0.002] ← both

Pick "mat" 50% of the times & "rug" 30% as both are good options.

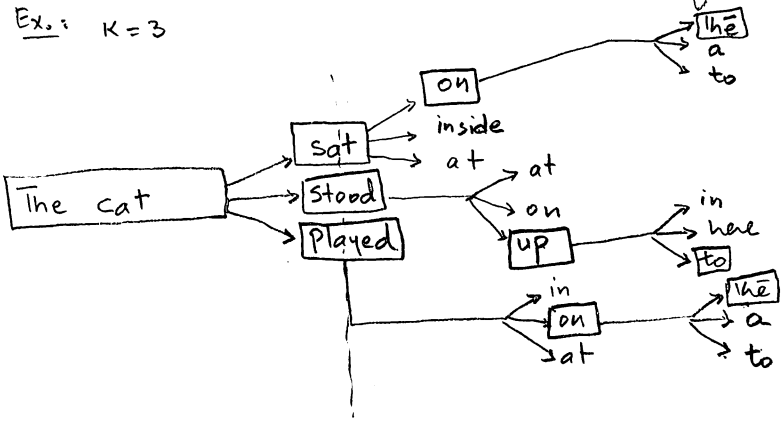
→ As $\rho \to 0$, it reduces to Greedy.

Problem: Low prob. tokens are sampled.

③ Beam Search: Explore multiple promising sequences simultaneously to find a global better sequence.

Beam Size = K → keep track of K most probable partial generations

Ex.: K=3



At step t, generate all possible extensions of K sequences, then select top-K sequences based on their cumulative probability score to carry-forward to next step.

Step $t \rightarrow \sum_{t=1}^{t} \log P_\theta(x_{i,t+1} \mid x_{i,\leq t})$

Score of i-th sequence. $i = 1, \ldots, K$

when K=1, it reduces to Greedy.

Problem: Beam search text is less surprising. Suffers from repetitive & uninteresting content. As LLMs become better, the lower-prob. tokens became better as well.

④ Top-K Sampling: Identify top-K tokens by prob. → renormalize & sample from them.

Our Example: K=3, "mat" — 0.5, "rug" — 0.3, "floor" — 0.15 → Renormalize → 0.5/0.95 → 0.526, 0.3/" → 0.316, 0.15/" → 0.158 → Naive Sampling

⑤ Top-$\hat{p}$ Nucleus Sampling:

Find smallest set of most likely tokens whose cumulative prob. $\geq \hat{p}$

Ex. $\hat{p} = 0.9$ "mat" → $0.5 < \hat{p}$

"mat" + "rug" → $0.5 + 0.3 = 0.8 < \hat{p}$

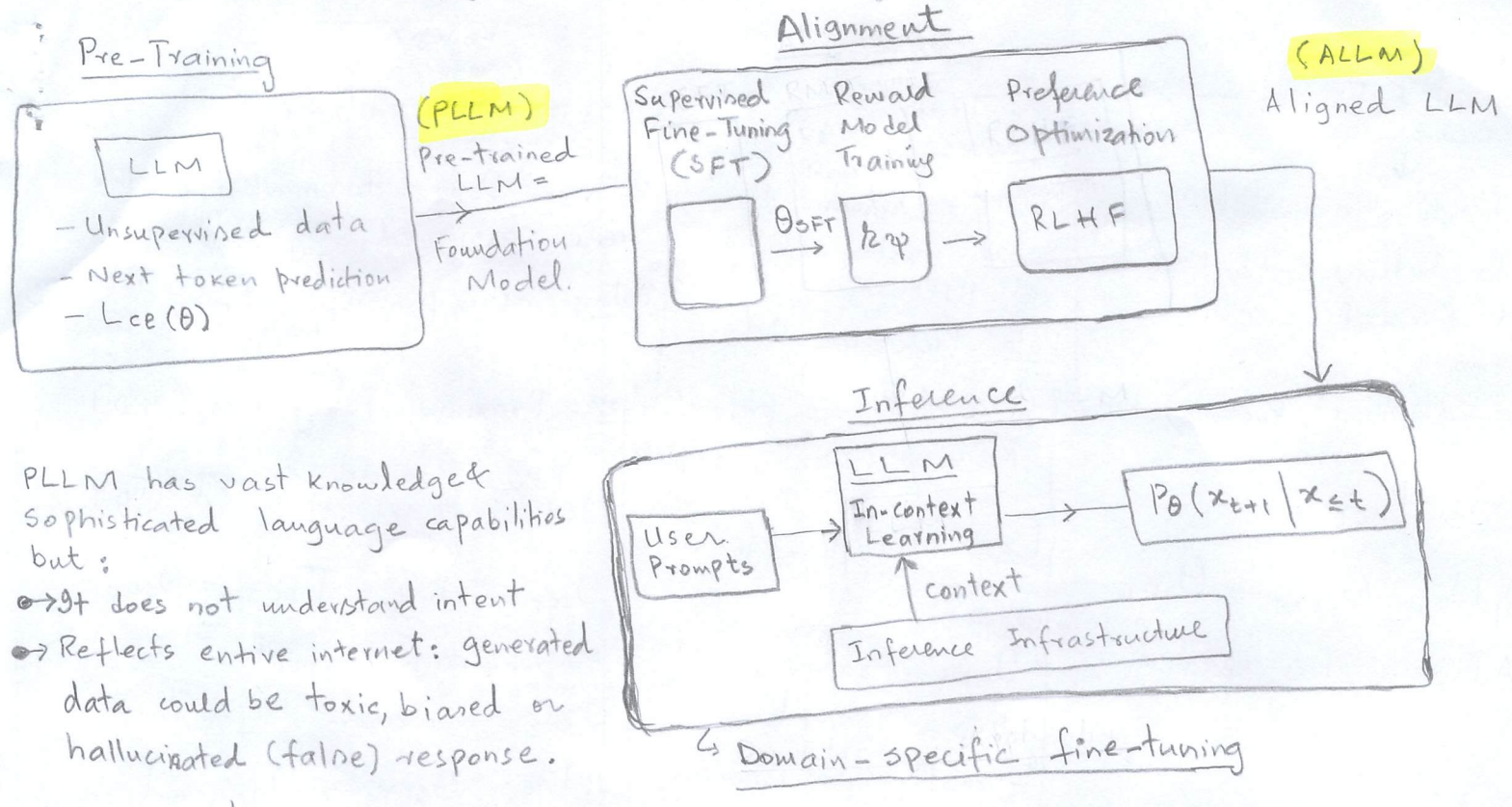
"mat" + "rug" + "floor" → $0.5 + 0.3 + 0.15 \geq \hat{p}$ → STOP

selected Nucleus = ["mat", "rug", "floor"] → Renormalize → Naive sampling

→ Modern LLMs use: combination of temperature ρ, top-K & top-$\hat{p}$.

differs from model to model. e.g. GPT uses ρ + top-$\hat{p}$

LLM's are trained & deployed in multiple stages:



PLLM has vast knowledge & sophisticated language capabilities but:

→ It does not understand intent

→ Reflects entire internet: generated data could be toxic, biased or hallucinated (false) response.

↓

PLLM is good but not useful:

To create a more useful model, we have to align PLLM to user's desires.

PLLM → Next Token = some text that comes on internet

↓

- Not controllable
- No instruction following
- Safety & Policy gaps in output

Alignment: Fine-tuning of PLLM to make its outputs helpful, harmless, and honest (HHH):

Helpful ↔ User Assistance to achieve the goals e.g. instruction following, relevance, completeness, clarity, Problem solving

Harmless ↔ Avoid causing emotional, social or physical harm to individuals or groups.

Honest ↔ Factuality + Truthfulness

Helpful	Harmless	Honest
Maximize model utility & effectiveness	Minimize model's potential to be misused.	Build user trust through reliable information

→ Foundational ethical & Safety framework

These 3 goals have inherent trade-offs:

① Helpful vs. Harmless: Refusal rate, accessibility of info.

② Honest vs. Helpful: Hallucination risk, "I do not know" → Honest / Unhelpful ?

③ Harmless vs. Honest: Factually correct answer might be potentially controversial / offensive

Most common alignment process is 3-Step process!

SFT → **RM Training** → **Policy/ Preference optimization**

SFT

Teach the model to follow instructions & format.

Demo.

Data: (Prompt, Response) pairs

Loss: L_{CE}

Update: $\theta_{LLM} \rightarrow \theta_{SFT}$

All parameters updated.

RM Training

Create AI-Judge to score model quality wrt HHH.

Data: Preference Data (Prompt, Response_preferred, Response_rejected) triplets.

Loss: Binary preference loss

Update:

θ_{SFT} = Frozen,

R_ψ = Trained.

separate model (smaller)

Policy/ Preference optimization

optimize θ_{SFT} to maximize the score assigned by R_ψ

Data: New Prompts / Data C Training Data

Loss: Proximal Policy loss (PPO)

$PPO = \text{Reward Maximization} + KLD(\pi_\theta \| \pi_{SFT})$

θ_{SFT} = Policy = π_{SFT} = θ_{SFT}

Environment = Prompt

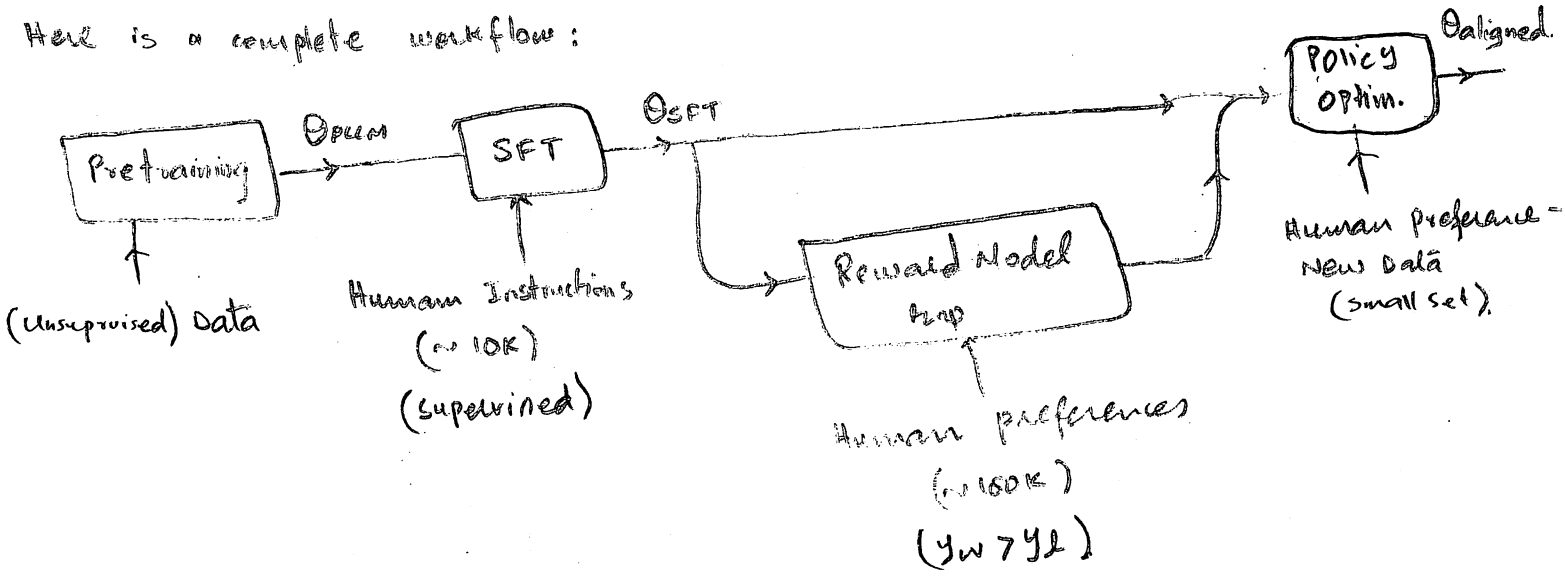
Score = Reward from R_ψ

Update: $\theta_{SFT} \rightarrow \theta_{aligned}$

Modern methods like DPO combine these two steps into

RLHF Alignment

Here is a complete workflow:



Supervised Fine-Tuning: New data is collected along with labels (correct Response)

Demonstration Data: Collect small data ~ 10K examples but of high quality.

↳ **Single-turn Format**:

Role	Example Text
Instruction/Prompt (user)	"Explain the theory of relativity"
Response (Assistant)	"The theory of relativity, developed by Albert Einstein, has two parts..."

↳ **Multi-turn/Conversational Format**:

Role	Example Text
SYSTEM	"You are a helpful & friendly science tutor"
USER	"What is the formula of Kinetic Energy"
MODEL	"The formula for kinetic energy is $KE = \frac{1}{2}mv^2$, where m is the mass and v is the velocity"
USER	"Can you give me an example"
MODEL	"Sure; If a 2kg ball is ..."

Loss function: $L_{SFT}(\theta) = -\sum_{t=1}^{T} \log P_\theta(x_{t+1} | x_{\leq t})$ → We use masking & Teacher-forcing.

However we use completion-only loss i.e. loss is calculated only over the tokens in the target response (Model/Assistant's completion). The tokens corresponding to USER/INSTRUCTION/SYSTEM PROMPT are masked-out.

↓ Serves as context ↔ Model is forced to learn to generate high-quality response & not repeat or overfit to user prompt or fixed system messages.

SFT training is run for a small no. of epochs: often between 1 & 5. More epochs can lead to overfitting on SFT data & catastrophic forgetting where model loses its vast general knowledge learned during pre-training.

PEFT: methods like LoRA, QLoRA, Prefix Tuning etc., can be used ↳ Not always during SFT due to compute efficiency, reduced memory footprint, and proven effectiveness in achieving comparable performance to full fine-tuning. Ex.: • GPT → Fully Fine-Tuned
• Llama, Mistral → PEFT

Ex: θ_{LLM} = Available , Demo. Data :

Ex. 1 : Input : "What is the capital of France?" (t_1 t_2 t_3 t_4 t_5 t_6 → t_m)

Target Response : "The capital of France is Paris". ($t_{m+1} \leftarrow t_7$ t_8 t_9 t_{10} t_{11} t_{12} → t_n)

PROMPT TOKENS

Tokenized Input : $t_1, t_2, \ldots, t_m, t_{m+1}, t_{m+2}, \ldots, t_n$ | ASSISTANT TOKENS

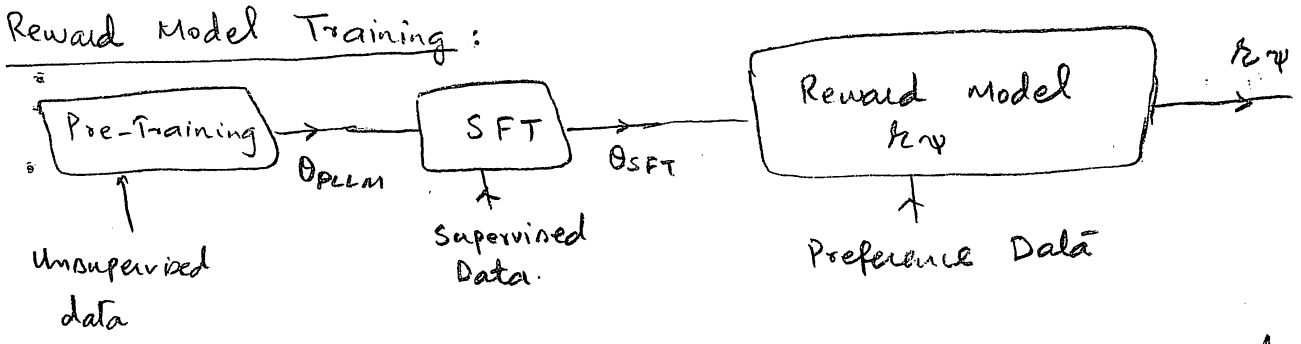
Mask : $-\infty, -\infty, \ldots, -\infty, 1, 1, \ldots, 1$

Labels y_t : $-\infty, -\infty, \ldots, -\infty, t_{m+2}, t_{m+3}, \ldots, \langle EOS \rangle$

$$L_{SFT}(\theta) = -\frac{1}{N_{valid}} \sum_{i=m+1}^{n-1} 1(y_t \neq -\infty) \log(P_{t, y_t})$$

$$\theta_{SFT} = \min L_{SFT}(\theta)$$

Reward Model Training :



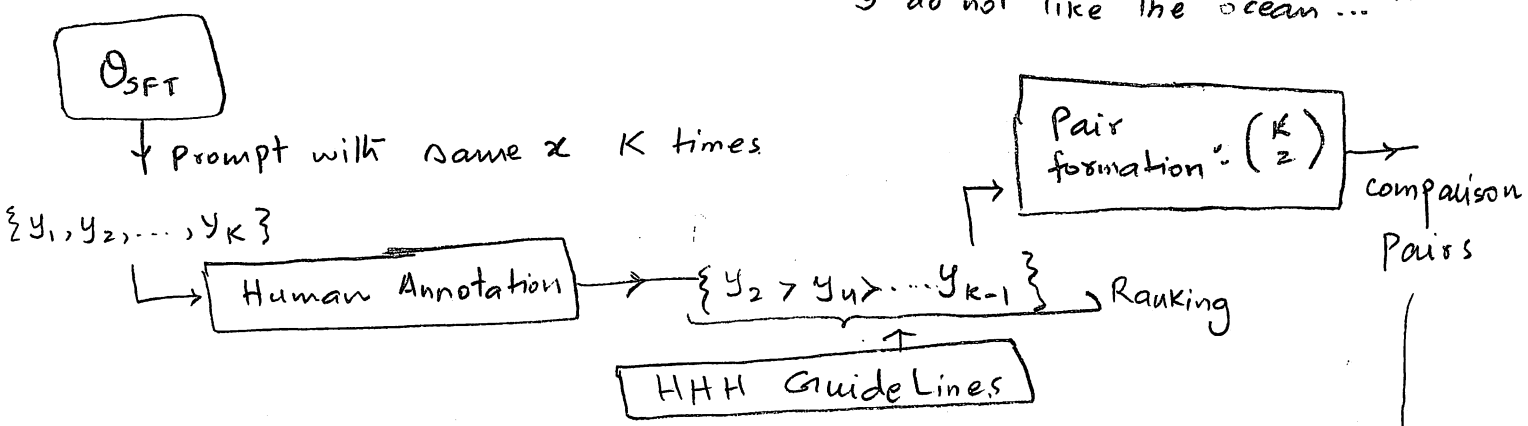
Preference data = sort-of supervised data having multiple answers to a prompt in a given example.

$D : \{x_i, y_{w_i}, y_{l_i}\}$ x_i = i-th prompt : "write a poem about ocean"

y_{w_i} = chosen / preferred / winner response "The ocean is vast and deep..."

y_{l_i} = Rejected / losing response "I do not like the ocean ..."

How to get it ?

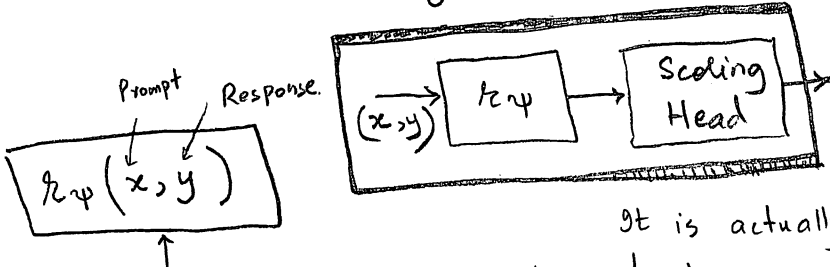


By batching like this helps in better generalization & reasoning.

Model r_ψ - A sequence to scalar function:

Initialize : $r_\psi = \theta_{SFT}$

Final Layer : Last layer is removed & a small linear-layer, called scaling head, is added.



$\hat{y} = P(y_w > y_l | x) = ?$ ← It is actually a probability that generated text is close to y_w & far from y_l

Bradley-Terry Model of preferences :

$$P(i > j) = \frac{p_i}{p_i + p_j}$$

→ Pairwise comparison prob. of two events drawn from same distribution, satisfy this relation

The prob. of event i being preferred over event j depends only on the ratio of their underlying strengths: $P(i>j) = \frac{\text{strength of } i}{\text{strength of } i + \text{" "} j}$

In our case:

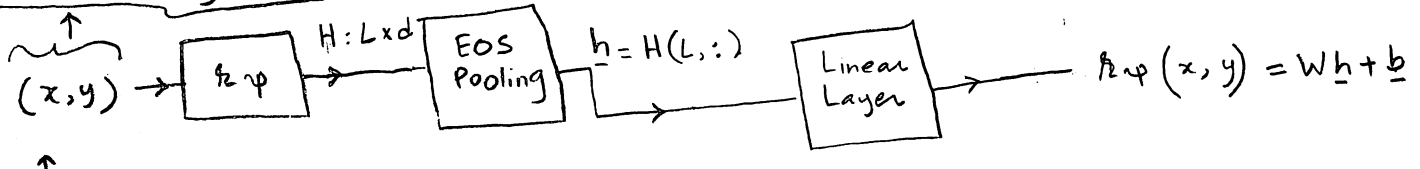
$$P_w = r_\psi(x, y_w)$$
$$P_l = r_\psi(x, y_l)$$
$$\Rightarrow P(y_w > y_l | x) = \frac{\exp(r_\psi(x, y_w))}{e^{r_\psi(x,y_w)} + e^{r_\psi(x,y_l)}}$$

$$\Rightarrow \log\left(\frac{P(y_w > y_l|x)}{P(y_l > y_w|x)}\right) = \underbrace{r_\psi(x, y_w) - r_\psi(x, y_l)}_{\hookrightarrow z = \text{Logit for our reward model training}}$$

$$L(\psi) = -\log(\sigma(z))$$

How do we get $r_\psi(x,y)$:

Prompt & its response concatenated & Parsed as single input



L - Tokens

[PROMPT_TOKEN] x [RESPONSE-TOKEN] y [EOS-TOKEN]

H = final layer output → also called Latent / hidden-state

d = embedding dimension of each token.

$\underline{h}$ = Summary vector = corresponds to [EOS-TOKEN]

⇒ The output vector for [EOS-TOKEN] is a weighted sum of entire input & has been informed by every single token of input.

We pass:

1- $(x, y_w) \longrightarrow r_\psi(x, y_w)$

2- $(x, y_l) \longrightarrow r_\psi(x, y_l)$

Then, make $L(\psi) = -\log\left(\sigma\left[\underbrace{r_\psi(x,y_w) - r_\psi(x,y_l)}_{=z}\right]\right) = -\log(z)$.

Finding a good reward model i.e. suitable $L(\psi)$ is relatively unexplored area. Some variants exist:

→ Reward model with preference margin → Llama 2 uses it

$L(\psi) = -\log(\sigma(z) - m(y_w, y_l))$ → Margin representing rank-difference b/w y_w & y_l.

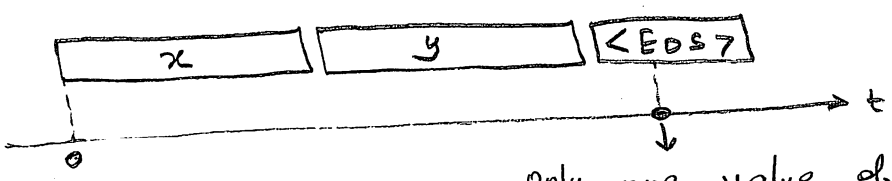
→ Multiple comparison per prompt:

$$L_{r_\psi} = -\frac{1}{\binom{K}{2}} E_{(x,y_w,y_l)\sim D_{\text{preference}}}\left[\log(\sigma(z))\right]$$

→ K-wise Loss function (Plackett-Luce Model): when full/partial ranking of K-responses is available → use it

$y_1 > y_2 > \ldots > y_K$.

$r_\psi(x,y)$ based on BT-model of preferences is very sparse reward.



Instantaneous reward = 0 except for last <EOS> token

only one value of reward for entire sequence.

There are other types which are used as well:

① Outcome-Reward-Model (ORM) with L_{CE}: Some tasks are inherently binary – correct or not-correct – e.g. Math problems, code execution result etc. For these tasks, we get preference data as: $D = \{(x, y, r)\}$ where x = Prompt, y = Response & $r \in [0,1] = \begin{cases} 1 & \text{if } y = \text{correct response} \\ 0 & \text{" } y \neq \text{ " " } \end{cases}$

Now, we can use BCE as loss:

$$L_{CE}(\psi) = -\mathop{E}_{(x,y,r)\sim D}\left[r \log\left(r_\psi(x,y)\right) + (1-r)\log\left(1-r_\psi(x,y)\right)\right]$$

$(x,y) \rightarrow \boxed{r_\psi} \rightarrow r_\psi(x,y)$ = Model's predicted prob. of correctness of y for prompt x

② Process Reward Model (PRM):

Used in reasoning-LLMs: PRM provides a dense step-by-step reward score to all intermediate steps.

$$L_{PRM}(\psi) = -\mathop{E}_{(x,s_i,y_{s_i})\sim D}\left[\sum_{i=1}^{K} y_{s_i} \log r_\psi(x,s_i) + (1-y_{s_i})\log\left(1-r_\psi(x,s_i)\right)\right]$$

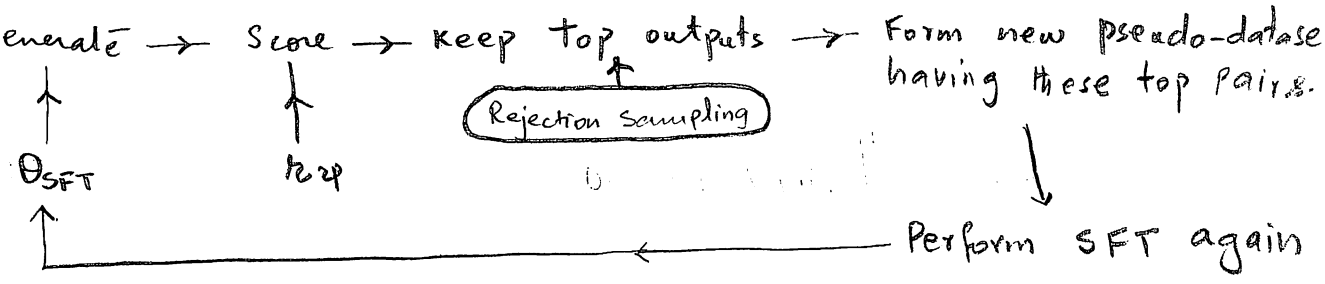
s_i = i-th reasoning step

$y_{s_i} \in [0,1]$ = label for s_i – Correct or not-Correct

$r_\psi(x,s_i)$ = Model's predicted prob that s_i = Correct step

Once r_ψ is trained, we move to alignment-preference fine-tuning stage. Usually it is done through:

→ Rejection Sampling: Select only the best outputs (completions) from θ_{SFT} based on r_ψ evaluations. Pipeline is:



- RS = Simple baseline to compare RLHF methods.

→ Reinforcement Learning with human feedback (RLHF): Learn policy via gradient updates using reward model. ⇒ Align with human Intent

→ RL with Verifiable Rewards (RLVR): use structured feedbacks - not just scalar rewards. RLVR replaces human-derived subjective reward with verifiable-objective reward (reward that can automatically be checked or proven correct). Example tasks include: code correctness, math accuracy, safety constraints etc. However it has limited coverage as applicable only where verification is possible.

⇒ Align with Ground-Truth Correctness